```
import cv2
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
import numpy as np


NAME: deepti
ROLL NO : 16

# 1. LOAD THE IMAGE

# The initial attempt to load 'image.jpg' might be redundant if an upload is expected.
# Keeping it here but the subsequent upload will override 'image' variable.
image = cv2.imread("image.jpg", cv2.IMREAD_COLOR)

from google.colab import files

uploaded = files.upload()

image_name = next(iter(uploaded))

# Load the uploaded image as a color image. Remove cv2.IMREAD_GRAYSCALE.
image = cv2.imread(image_name, cv2.IMREAD_COLOR)

if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully.")
    print("Image size:", image.shape)

NAME: deepti
ROLL NO : 16

# 2. DISPLAY IMAGE USING OPENCV (Colab compatible)

cv2_imshow(image)

# cv2.waitKey(0)
# cv2.destroyAllWindows()

NAME: deepti
ROLL NO : 16



# 3. DISPLAY IMAGE USING MATPLOTLIB


# Convert BGR to RGB
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.imshow(rgb_image)
plt.title("Original Image")
plt.axis("off")
plt.show()

NAME: deepti
ROLL NO : 16


# 4. EXAMINE IMAGE PROPERTIES
height, width, channels = image.shape

print("Image Height:", height)
print("Image Width:", width)
print("Number of Channels:", channels)
print("Image Data Type:", image.dtype)
# height, width = image.shape[:2]

# if len(image.shape) == 3:
#     channels = image.shape[2]
# else:
#     channels = 1

# print("Image Height:", height)
# print("Image Width:", width)
# print("Number of Channels:", channels)
# print("Image Data Type:", image.dtype)

NAME: deepti
ROLL NO : 16
```

```
# 5. SAVE IMAGE IN JPEG AND PNG FORMATS


cv2.imwrite("output.jpg", image)
cv2.imwrite("output.png", image)

print("Image saved as JPEG and PNG successfully!")
NAME: deepti
ROLL NO : 16



# 6. CONVERT IMAGE TO GRAYSCALE


gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()

NAME: deepti
ROLL NO : 16


# 7. CONVERT IMAGE TO HSV


hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV Image")
plt.axis("off")
plt.show()

NAME: deepti
ROLL NO : 16

# 8. CONVERT IMAGE TO LAB


lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB))
plt.title("LAB Image")
plt.axis("off")
plt.show()
NAME: vansh
ROLL NO : 64



# 9. RESIZE IMAGE


resized = cv2.resize(image, (200, 300))

plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized Image")
plt.axis("off")
plt.show()

NAME: deepti
ROLL NO : 16


# 10. ROTATE IMAGE


height, width = image.shape[:2]

center = (width // 2, height // 2)

matrix = cv2.getRotationMatrix2D(center, 45, 1.0)

rotated = cv2.warpAffine(
    image,
    matrix,
    (width, height)
)

plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image")
plt.axis("off")
```

```
plt.show()
```

NAME: deepti
ROLL NO : 16

```
# 11. HORIZONTAL AND VERTICAL FLIPPING


horizontal_flip = cv2.flip(image, 1)

vertical_flip = cv2.flip(image, 0)

plt.figure(figsize=(4, 3))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
plt.title("Vertical Flip")
plt.axis("off")

plt.show()
```

NAME: deepti
ROLL NO : 16

```
# 12. GENERATE NEGATIVE IMAGE


negative = 255 - image

plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative Image")
plt.axis("off")
plt.show()
```

NAME: vansh
ROLL NO : 64

```
# 13. EXTRACT REGION OF INTEREST (ROI)


roi = image[150:500, 100:350]

plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Region of Interest (ROI)")
plt.axis("off")
plt.show()

# Analyze ROI properties
print("ROI Shape:", roi.shape)
print("ROI Height:", roi.shape[0])
print("ROI Width:", roi.shape[1])
print("ROI Channels:", roi.shape[2])
```

NAME: deepti
ROLL NO : 16

```
# 14. FINAL COMPARISON


plt.figure(figsize=(4, 3))

plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized")
plt.axis("off")

plt.subplot(2, 3, 4)
```

```
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative")
plt.axis("off")

plt.tight_layout()
plt.show()

NAME: deepti
ROLL NO : 16



# """final Observations:-

# #| Operation        | Observation                                                                    |
# #| ---------------- | ------------------------------------------------------------------------------ |
# #| Image loading   | Color image was successfully loaded using OpenCV.                               |
# #| Image properties | Image size was **739*415 pixels**, with **3 channels** and `uint8` data type.   |
# #| JPEG/PNG        | The image was successfully saved in both JPEG and PNG formats.                  |
# #| Grayscale        | Color information was converted into a single intensity representation.         |
# #| HSV              | Image was represented using Hue, Saturation and Value components.               |
# #| LAB              | Image was represented using Lightness and A/B color components.                 |
# #| Resizing         | Image dimensions were changed to **500*500** pixels.                            |
# #| Rotation        | Image was rotated by **45°**.                                                    |
# #| Flipping         | Horizontal and vertical mirror transformations were successfully performed.     |
# #| Negative        | Pixel intensities were inverted using `255 - image`.                             |
# #| ROI              | A selected portion of the original image was extracted and analyzed.            |



#  QUESTIONS:

# 1. What is a digital image? Differentiate between grayscale and color images.

# Answer:
# A digital image is a two-dimensional representation of a visual scene in the form of pixels. Each pixel contains numerical information re

# Grayscale image:

# Contains only intensity information.
# Usually has one channel.
# Pixel values generally range from 0 to 255.
# 0 represents black and 255 represents white.

# Color image:

# Contains color information.
# Commonly represented using three channels such as Red, Green, and Blue (RGB).
# Each pixel contains three intensity values corresponding to the three color channels.


# 2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

# Answer:

# RGB: Represents an image using Red, Green, and Blue channels. It is commonly used for displaying images.
# BGR: Uses Blue, Green, and Red channels in that order. OpenCV generally reads color images in BGR format.
# HSV: Represents color using Hue, Saturation, and Value. It is useful for color-based image processing because color and brightness are re
# LAB: Represents an image using L (Lightness), A, and B components. L represents lightness, while A and B represent color information.

# Thus, different color spaces represent the same image differently and are useful for different image-processing tasks.



# 3. What is the purpose of converting an image to grayscale before further processing?

# Answer:
# Converting an image to grayscale removes color information and represents the image using intensity values. This reduces the amount of da

# Grayscale conversion is useful because:

# It reduces a three-channel color image to a single channel.
# It simplifies image processing.
# It reduces computational requirements.
# It is useful for operations such as edge detection, thresholding, and feature extraction.

#    Therefore,    grayscale    conversion    is    commonly    used    as    a    preprocessing    step    in    computer    vision.
```

```
# 4. Explain the concept of image complement (negative image) and mention its practical applications.

# Answer:
# An image complement, or negative image, is produced by reversing the intensity values of the pixels.

# For an 8-bit image:

# Negative pixel = 255 - Original pixel

# For example:
# Original pixel = 50
# Negative pixel = 255 - 50 = 205

# Similarly, black becomes white and white becomes black.

# Applications include:

# Medical image analysis.
# Improving visibility of certain image details.
# Analysis of photographic negatives.
# Document and image enhancement.
# Highlighting features that may be difficult to see in the original image.



# 5. Differentiate between image resizing, cropping, and scaling.

# Answer:

# Operation Meaning
# Resizing  Changes the dimensions of an image, such as converting it from 739*415 to 500*500 pixels.
# Cropping  Removes unwanted portions of an image and keeps only a selected region.
# Scaling     Changes the size of an image by multiplying its dimensions by a scale factor.

# For example, if an image is scaled by a factor of 2, its width and height are increased proportionally.


# 6. What is a Region of Interest (ROI), and why is it important in computer vision?

# Answer:
# A Region of Interest (ROI) is a selected portion of an image that contains the important information required for further processing.

# For example, instead of processing an entire photograph, we can select only the region containing a particular object.

# ROI is important because:

# It reduces the amount of data to process.
# It improves computational efficiency.
# It focuses processing on relevant areas.
# It can improve the accuracy of object detection and feature extraction.

# ROI is commonly used in object detection, medical imaging, face detection, and industrial inspection.

# 7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

# Answer:
# OpenCV is widely used for computer vision because it provides a large collection of optimized functions for image and video processing.

# Advantages include:

# Supports image reading, writing, and manipulation.
# Provides functions for color-space conversion.
# Supports geometric transformations.
# Provides tools for filtering, edge detection, and feature extraction.
# Supports video processing and real-time computer vision.
# Works efficiently with NumPy arrays.
# Provides many algorithms required for modern computer vision applications.

# Therefore, OpenCV provides a comprehensive framework for implementing computer vision applications.



# 8. Explain how image resolution and pixel intensity influence image quality.

# Answer:

# Image resolution refers to the number of pixels used to represent an image. Higher resolution generally provides more detail because more

# For example:

# Low resolution  → fewer pixels → less detail
# High resolution → more pixels  → more detail

# Pixel intensity represents the brightness of a pixel. In an 8-bit grayscale image:

# 0   → Black
```

```
# 255 → White

# Intermediate values represent different shades of gray.

# Therefore, resolution affects the amount of spatial detail, while pixel intensity affects the brightness information of the image.



# 9. Mention five real-world applications where basic image preprocessing is an essential step.

# Answer:
# Five real-world applications are:

# Medical imaging – preprocessing X-rays, CT scans, and MRI images.
# Autonomous vehicles – preparing camera images for object and road detection.
# Face recognition – preprocessing facial images before feature extraction.
# Industrial automation – inspecting products for defects.
# Surveillance systems – improving images before object or activity detection.

# These preprocessing operations help prepare raw images for subsequent computer vision tasks.



# 10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

# Answer:
# Image preprocessing prepares raw images so that important information can be extracted more effectively.

# Preprocessing can:

# Remove unwanted or irrelevant information.
# Standardize image dimensions.
# Convert images into suitable color spaces.
# Improve image quality.
# Reduce unnecessary computational complexity.
# Focus processing on important regions using ROI.
# Provide consistent input to machine-learning and deep-learning models.

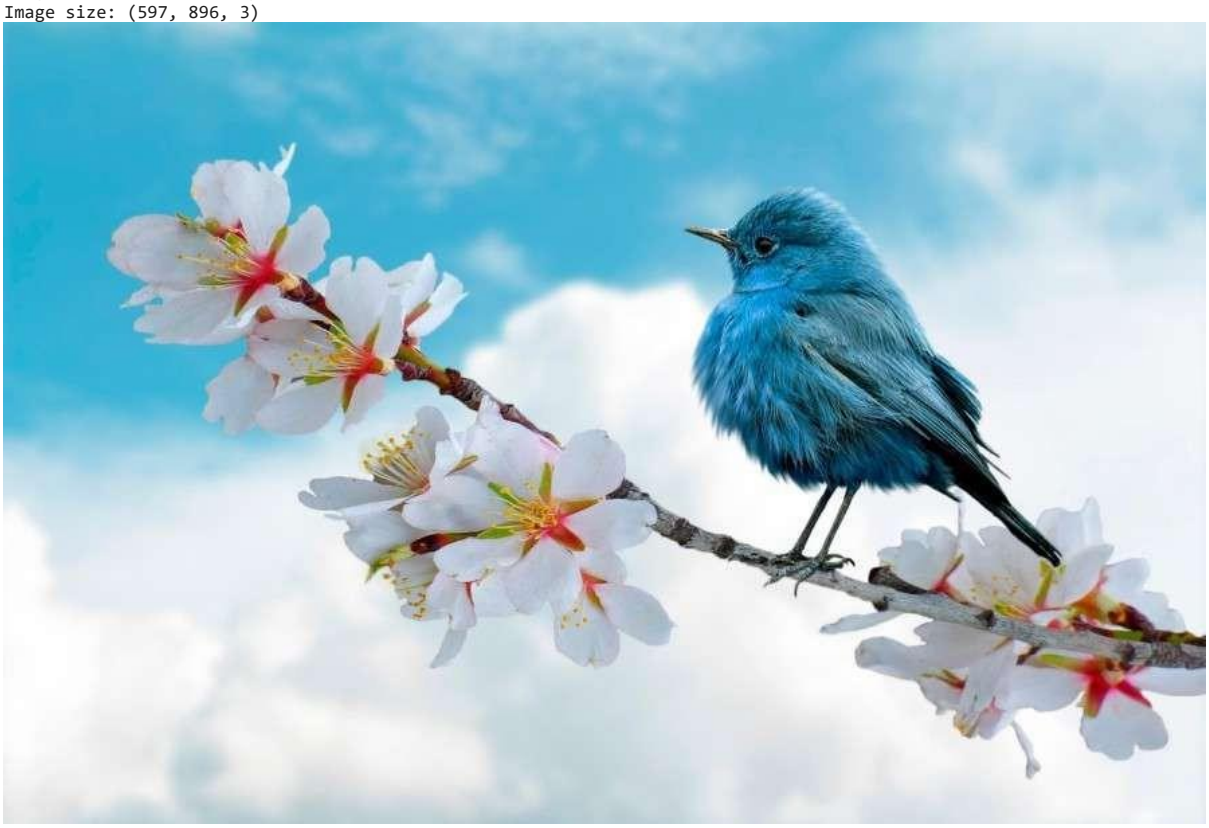
# For example, resizing ensures that images have a consistent input size, while grayscale conversion can reduce unnecessary color informati

# As a result, preprocessing can help feature extraction and deep-learning models work more efficiently and can improve the quality and con
```

Choose files bird-8788491_1280.jpg

bird-8788491_1280.jpg(image/jpeg) - 81658 bytes, last modified: 20/08/2026 - 100% done

```
Saving bird-8788491_1280.jpg to bird-8788491_1280 (7).jpg
Image loaded successfully.
Image size: (597, 896, 3)
```



Original Image



```
Image Height: 597
Image Width: 896
Number of Channels: 3
Image Data Type: uint8
Image saved as JPEG and PNG successfully!
```

Grayscale Image



HSV Image





LAB Image

